



Calango

Packaging Guide

Building the macOS `.dmg`
and Linux `.deb` installers

Leandro Seixas

Calango 26.8.18

Abstract

Calango ships as two native installers built with CMake/CPack: a drag-and-drop disk image (`.dmg`) for macOS and a Debian package (`.deb`) for Ubuntu/Debian. This guide covers the prerequisites, the exact commands, what the packaging pipeline does under the hood, how to verify the artifacts, and the failure modes we have already hit — with their fixes. Everything described here lives in `CMakeLists.txt` (install and CPack sections) and in the `packaging/` directory of the repository.

Contents

1 Overview	1
2 macOS: building the <code>.dmg</code>	2
2.1 Prerequisites	2
2.2 Commands	2
2.3 The resulting image	3
2.4 Cleaning up afterwards	3
2.5 What the install step does	4
2.6 Verification	4
3 Linux: building the <code>.deb</code>	4
3.1 Prerequisites (Ubuntu/Debian)	4
3.2 Commands	5
3.3 Package contents and desktop integration	5
3.4 Verification	6
4 Conda package	6
4.1 Commands	6
4.2 What the recipe pulls in	6
5 The embedded Python environment	7
6 Known limitations and next steps	7

1 Overview

Both installers come out of the standard CMake flow: `configure` → `build` → `cpack`. The platform decides the generator:

Platform	CPack generator	Artifact
macOS (<code>-DCALANGO_MACOS_BUNDLE=ON</code>)	DragNDrop	<code>calango-26.8.18-macos-arm64.dmg</code>
Linux	DEB	<code>calango_26.8.18_amd64.deb</code>

Each artifact is accompanied by a `.sha256` checksum file. The package version comes from `project(calango VERSION ...)` in `CMakeLists.txt` — the single source of truth for the version, baked into the binary as `CALANGO_VERSION` (About dialog, project files) and reused by CPack for the installer file names.

The supporting files are:

- `packaging/macos/make_icns.sh` — renders `calango.icns` from the master PNG icon in `assets/calango/` (used when no committed `.icns` exists).
- `packaging/macos/embed_python_framework.sh` — makes the `.app` independent of the build machine's Python (Section 5).
- `packaging/linux/calango.desktop` — application launcher entry.
- `packaging/linux/calango-mime.xml` — registers the `application/x-calango-project` MIME type for `.calproj` project files.
- `packaging/linux/postinst`, `postrm` — refresh the desktop/MIME/icon caches on install and removal.

2 macOS: building the `.dmg`

2.1 Prerequisites

- Xcode command-line tools (provides `sips`, `iconutil`, `codesign`, `hdiutil`).
- Qt 6 with the Svg module; Homebrew's `brew install qt` puts it at `/opt/homebrew/opt/qt`.
- CMake ≥ 3.21 .
- A Python virtualenv with ASE installed, e.g. the project `.venv` (`python3 -m venv .venv` $\hookrightarrow$ `&& .venv/bin/pip install ase`).

2.2 Commands

The supported one-shot route is the driver script, which provisions the relocatable Python payload, builds, packages, and mounts the finished image to prove the shipped app can import ASE:

```
packaging/macos/create_dmg.sh
```

It also picks the build directory for you, which matters — see the warning below. To drive the steps by hand instead, from the repository root:

```
BUILD=~/.Library/Caches/calango/build-dmg

cmake -S . -B $BUILD \
  -DCMAKE_BUILD_TYPE=Release \
  -DPython3_EXECUTABLE=$PWD/.venv/bin/python \
  -DCMAKE_PREFIX_PATH=/opt/homebrew/opt/qt \
  -DCALANGO_MACOS_BUNDLE=ON

cmake --build $BUILD -j 8

cd $BUILD && cpack
```

The staging tree must not be cloud-synced. A file provider (Dropbox, iCloud Drive, OneDrive, Google Drive) stamps its own extended attributes on every file it manages, and re-applies them faster than they can be cleared. `codesign` rejects those as “*resource fork, Finder information, or similar detritus*”. Since the checkout itself lives in Dropbox on this machine, a build directory named relative to it (`build-dmg`) puts `_CPack_Packages/` inside the synced

tree, and the run fails at the very last step — after the full build, `macdeployqt` and the Python embed — with

```
calango.app: resource fork, Finder information, or similar detritus not allowed
error: calango.app fails strict signature validation
```

The fix is to stage outside the synced folder, as above. `create_dmg.sh` does this automatically: a `BUILD_DIR` you set yourself is refused with an explanation, and the default is relocated to `~/Library/Caches/calango/` with a notice. An already built tree does not need reconfiguring — `cpack -B` moves only the staging directory and the output:

```
cd build-dmg && cpack -G DragNDrop -B ~/Library/Caches/calango/cpack-stage
```

A finished `.dmg` is a single signed file that syncing cannot spoil, so only the staging tree has to move.

2.3 The resulting image

The result is `calango-<version>-macos-arm64.dmg`. Mounting it shows `calango.app` next to an `/Applications` symlink — the standard drag-to-install layout. There is deliberately no click-through license sheet (`CPACK_DMG_SLA_USE_RESOURCE_FILE_LICENSE OFF`): a license SLA blocks scripted `hdiutil attach` and adds friction for an MIT-licensed application.

2.4 Cleaning up afterwards

The scratch tree is several GB — a Release build of the application, the CPack staging copy of the bundle, and the provisioned Python payload — and every byte of it is reproducible. `create_dmg.sh` therefore deletes it as its last step, once the image has been built, mounted and probed:

```
==> SUCCESS
Artifact : /path/to/Calango-<version>-macOS.dmg
Size      : 65M
Verified  : valid macOS disk image (hdiutil imageinfo)
Probe     : python: 3.14.6 ase: 3.29.0
Cleaned   : ~/Library/Caches/calango/macos-bundle-a1b2c3d4 (freed 3.4G)
           next run re-provisions the embedded Python; --keep-cache avoids that
```

Three properties of that step are deliberate:

- **It runs last, and only on success.** A run that failed anywhere keeps its tree, because that tree is exactly what is needed to diagnose the failure.
- **It deletes the per-checkout subdirectory, not the root.** The scratch directory is keyed by the repository's path (`macos-bundle-<hash>`) so that several checkouts can coexist; removing `~/Library/Caches/calango` wholesale would take another checkout's cache with it. The root itself is then removed only if it is left empty.
- **It only ever deletes inside that root.** A `BUILD_DIR` pointing anywhere else is reported and kept — it may be a working build directory that predates the packaging run.

Pass `--keep-cache` while iterating on packaging itself. The tree caches the ~500 MB embedded-Python payload, so a kept tree makes the following run skip the download and reuse the compiled objects as well. Cleaning up by hand after a manual `cpack` is the same idea:

```
rm -rf ~/Library/Caches/calango/cpack-stage
```

2.5 What the install step does

cpack internally runs the CMake install rules into a staging directory (`_CPack_Packages/`) and then wraps the result in a disk image. The install rules perform, in order:

1. **Stage the bundle.** `calango.app` is installed with the Finder icon (`calango.icns`, committed or rendered at build time by `make_icns.sh`) and the `Info.plist` metadata (bundle id `org.seixasresearch.calango`).
2. **Optionally embed a Python environment** into `Contents/Resources/python`, when the cache variable `CALANGO_EMBEDDED_PYTHON_DIR` is set (Section 5).
3. **Deploy Qt with `macdeployqt` — twice.** The tool copies the Qt frameworks and plugins into the bundle and rewrites their load paths to `@executable_path`. Two passes with `-libpath=/opt/homebrew/opt/qt/lib` are required: frameworks copied into the bundle lose the `@loader_path` context their `@rpath` dependencies resolve against, so transitive dependencies (e.g. `QtGui` → `QtDBus`, plugin dependencies) are only found when the second pass re-scans the already-copied frameworks. The first pass prints a wall of `ERROR: Cannot resolve rpath` lines — this is expected noise; judge success by the verification steps below.
4. **Embed the linked `Python.framework`.** `embed_python_framework.sh` copies the framework version the binary links against (typically Homebrew's) into `Contents/Frameworks`, replaces Homebrew's dangling `site-packages` symlink with a real directory, recreates the canonical framework symlinks, retargets the load command with `install_name_tool`, and finally re-signs the whole bundle ad hoc (`codesign --force --deep --sign -`). Without the re-sign, Apple-silicon macOS refuses to load the modified binaries.

2.6 Verification

```
APP=_CPack_Packages/Darwin/DragNDrop/calango-<version>-macos-arm64/calango.app

# 1. Signature must verify strictly, including nested code
codesign --verify --deep --strict "$APP"

# 2. The packaged binary must start and find a Python with ASE
"$APP/Contents/MacOS/calango" --probe-python

# 3. The image must mount without an SLA prompt
hdiutil attach -readonly -nobrowse calango-<version>-macos-arm64.dmg
```

To rebuild the package from a clean state, clear the staging area first with `cmake -E rm -rf ↵ _CPack_Packages` — stale staging can mask deployment bugs.

3 Linux: building the .deb

3.1 Prerequisites (Ubuntu/Debian)

```
sudo apt install build-essential cmake ninja-build \
  qt6-base-dev libqt6svg6-dev libqt6opengl6-dev \
  libgl1-mesa-dev python3-dev python3-venv \
  dpkg-dev file
```

`dpkg-dev` provides `dpkg-shlibdeps`, which CPack uses to compute the runtime `Depends:` line

automatically (CPACK_DEBIAN_PACKAGE_SHLIBDEPS ON).

Build against the system Python, not Conda/Homebrew. The `.deb` must link the distribution's `libpython` (`/usr/lib/.../libpython3.x.so.1.0`, owned by the `libpython3.x` package) so that `dpkg-shlibdeps` can resolve it into a proper `Depends:` line. If CMake is pointed at a Conda or Homebrew interpreter, the binary links a `libpython` under `$HOME` that `dpkg-shlibdeps` cannot find — packaging fails with `dpkg-shlibdeps: error: cannot find library` — and the resulting package would not be installable on other machines anyway. `python3-dev` installs the system interpreter and its `libpython`; configure against `/usr/bin/python3` (Section 3.2). ASE is only a runtime `Recommends` (`apt install python3-ase`); it is not needed to build the package.

3.2 Commands

Configure a *fresh*, Linux-native build directory — never reuse a tree configured on another machine or OS (see the note below) — and point CMake at the system interpreter:

```
cmake -S . -B build-deb \
  -DCMAKE_BUILD_TYPE=Release \
  -DPython3_EXECUTABLE=/usr/bin/python3

cmake --build build-deb -j$(nproc)

cd build-deb && cpack
```

Do not run `cpack` in a build directory copied from another host. Cloud-synced check-outs (Dropbox) carry the macOS build/ tree along with the source. Its `CMakeCache.txt` and `CPackConfig.cmake` bake in absolute macOS tool paths, so a Linux `cpack` there fails with `CPack Error: Cannot find description file name: [/opt/homebrew/share/cmake/Templates/...]`. A separate `build-deb` directory, configured by the Linux `cmake`, avoids this entirely.

The result is `calango_26.8.18_<arch>.deb`. Install it via `apt` rather than `dpkg -i`, so the dependencies are resolved:

```
sudo apt install ./calango_<version>_amd64.deb
```

3.3 Package contents and desktop integration

Path in package	Purpose
<code>/usr/bin/calango</code>	The application binary
<code>/usr/share/applications/calango.desktop</code>	Launcher entry (<code>Exec=calango %F</code>)
<code>/usr/share/mime/packages/calango-mime.xml</code>	MIME type for <code>.calproj</code>
<code>/usr/share/pixmaps/calango.png</code>	Application icon (resolves the <code>.desktop Icon=calango</code> lookup)
<code>/usr/lib/calango/python/</code>	Embedded Python (optional, see Section 5)

The MIME registration means double-clicking a `.calproj` workspace in the file manager opens it in Calango; `MainWindow::loadFile` routes `.calproj` arguments to the project loader rather than the ASE structure reader. The `postinst/postrm` maintainer scripts refresh `update-mime-database`, `update-desktop-database` and `gtk-update-icon-cache` so the association is live immediately after install (modern `dpkg` triggers usually do this anyway; the scripts make it robust on minimal systems).

Unless an embedded Python is bundled, the package declares `Depends: python3 (>= 3.9)` and `Recommends: python3-ase`. Without ASE the GUI starts but structure I/O and job submission are disabled; users can also point Calango at any interpreter via `CALANGO_PYTHON`.

3.4 Verification

```
dpkg-deb --info calango_<version>_amd64.deb # control fields, Depends
dpkg-deb --contents calango_<version>_amd64.deb

# after installing:
calango --probe-python
xdg-mime query default application/x-calango-project
```

Status note: the DEB pipeline has been built and verified end-to-end on Ubuntu (`dpkg-deb -info` shows the auto-computed `Depends:`, including `libpython3.12t64` and the Qt 6 runtime libraries). Running it in CI on a clean Ubuntu 22.04/24.04 image remains worthwhile to catch host-specific assumptions.

4 Conda package

The Conda recipe lives in `packaging/conda/` and differs from the two installers above in kind: it does not ship a bundled interpreter. Conda *is* the environment, so Calango declares its Python stack as ordinary runtime dependencies and the solver provides them.

4.1 Commands

```
# From the repository root
bash packaging/conda/create_conda_osx-arm64.sh

# Result: packaging/conda/dist/calango-<version>-<build>.conda
```

The script finds whichever build tool is available — `conda-build`, `boa` (`conda mambabuild`), or `rattler-build` when a `recipe.yaml` is present — validates the recipe, and emits the modern `.conda` format into `packaging/conda/dist/`. Useful overrides:

Variable	Effect
<code>CHANNELS</code>	Extra channels; defaults to <code>-c conda-forge</code> .
<code>OUTPUT_DIR</code>	Where the artifact lands; defaults to <code>dist/</code> .
<code>CONDA_SOLVER</code>	Defaults to <code>libmamba</code> . The classic solver fails this recipe on the platform virtual packages (<code>__osx</code> , <code>__unix</code> , <code>__archspec</code>).
<code>CALANGO_VERSION</code>	Overrides the version in <code>meta.yaml</code> ; keep it in step with <code>CMakeLists.txt</code> .

4.2 What the recipe pulls in

`host` carries only what Calango compiles against — `qt6-main`, `pybind11` and `python`. Everything the embedded interpreter imports at run time is in `run`: `ase`, `numpy`, `scipy`, `spglib`, `pymatgen`, `paramiko`, plus `pymace` for machine-learning potentials.

Apple Silicon caveat. `gpaw` is gated on `[x86_64]`: `conda-forge` does not build GPAW for `osx-arm64`, so on Apple Silicon it must be installed into the environment separately with `pip install gpaw`. `gpaw-data` (the PAW datasets) *is* available on every platform and is always

pulled in. Every DFT workflow in the user guide depends on this, so it is worth checking before concluding that a wizard is broken.

The recipe's only test is `calango --probe-python`, which exits non-zero unless the interpreter it resolves can import ASE — a cheap end-to-end check that the packaged binary and the packaged Python stack actually agree.

5 The embedded Python environment

Calango embeds CPython at runtime. The interpreter is resolved in this order (documented in `src/python_bridge/PythonEngine.hpp`):

1. `$CALANGO_PYTHON` — explicit interpreter path;
2. `$VIRTUAL_ENV/bin/python` — active virtualenv;
3. a Python bundled by the installers: `Contents/Resources/python/bin/python3` (macOS) or `<prefix>/lib/calango/python/bin/python3` (Linux);
4. the interpreter CMake was configured against (`CALANGO_DEFAULT_PYTHON`, meaningful only on the build machine).

To ship a fully self-contained installer, pass a *relocatable* Python distribution that has ASE installed:

```
cmake -S . -B $BUILD ... \  
-DCALANGO_EMBEDDED_PYTHON_DIR=/path/to/standalone-python
```

A distribution from the `python-build-standalone` project is the recommended source. A plain project `.venv` is *not* relocatable — its `bin/python` and `pyvenv.cfg` refer back to the build machine's base interpreter.

`create_dmg.sh` does all of this for you when the variable is unset. It downloads a matching `python-build-standalone` tree, pip-installs `CALANGO_EMBEDDED_PACKAGES` into it, and caches the result in the build directory. Skipping this is what separates a `.dmg` that only runs on the build machine from a shippable one — the app still launches either way, it just cannot import ASE — and the mount-and-probe check at the end of the script is what tells the two apart.

Independent of this option, the macOS pipeline always embeds the `Python.framework` that the binary itself links against (`libpython` via `Development.Embed`), so the packaged app launches on machines without Homebrew. For the interpreter to also find ASE there, either bundle an environment as above or let users install ASE and set `CALANGO_PYTHON`.

6 Known limitations and next steps

- **Code signing:** the `.app` is signed ad hoc. Distribution outside a lab requires a Developer ID certificate and notarization (`codesign -sign "Developer ID..."`, `notarytool submit, stapler`); the ad-hoc step in `embed_python_framework.sh` is the place to swap the identity in.
- **DEB validation:** run the Linux pipeline once in CI (Ubuntu 22.04/24.04) and consider lintian for policy checks.

- **Full ASE bundling:** wiring a `python-build-standalone` + ASE tree into both installers (and building against its `libpython`) would remove the last runtime dependency on a user-provided Python.